Software Engineering

CSC648/848

SW Engineering Process Components

1. *Requirements and Specification*
2. *Design and Implementation*
3. *Validation*
4. *Maintenance and Evolution*

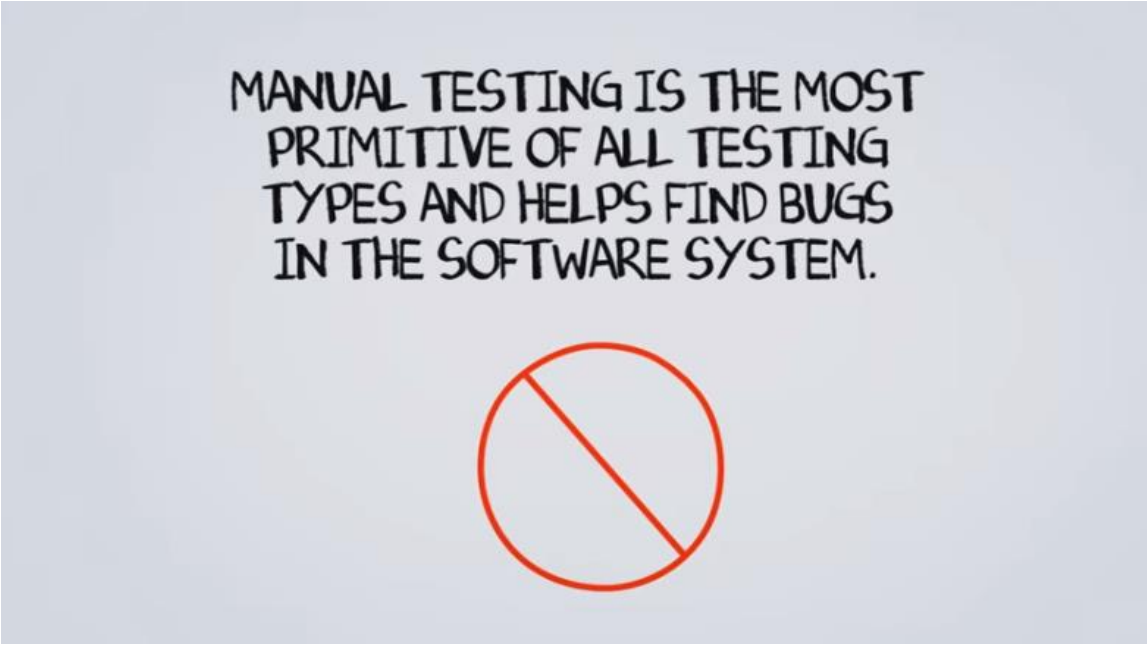
Henry
VILLAR

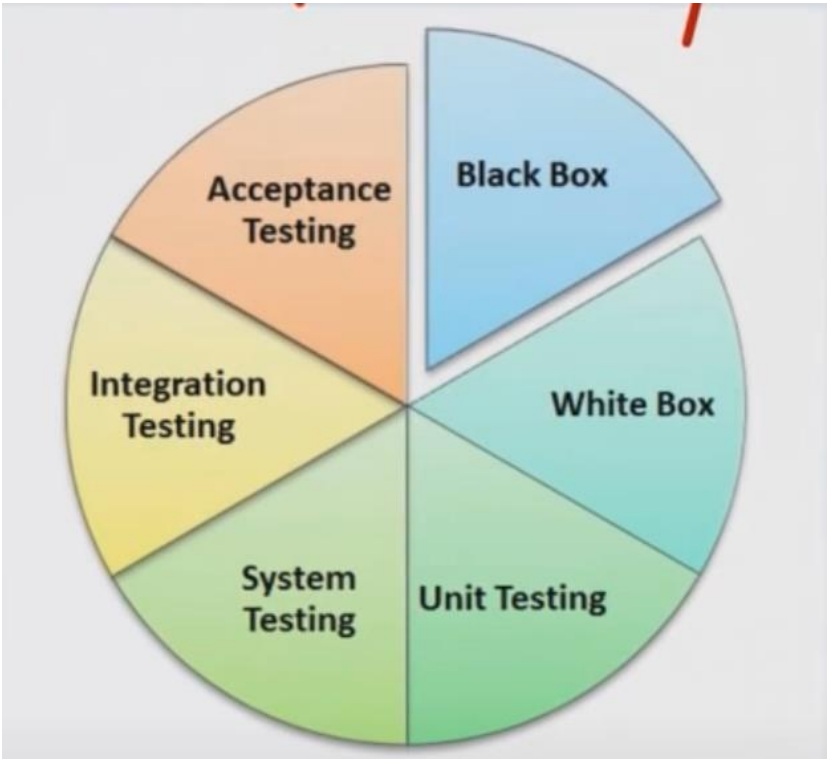
SW QUALITY ASSURANCE

Testing and QA

Remember CSC 639 Software Testing and Quality Assurance?

- Software system quality components and activities that support software quality • QA objectives: reliability, correctness, testability, maintainability, flexibility, portability, efficiency, integrity, usability, reusability, and interoperability. • Theoretical background: program correctness proofs, cyclomatic complexity, software reliability modeling. • Software unit testing to verify unit specifications • Integration testing to verify design specifications • System testing to verify requirements specifications • Usability testing • Performance, reliability, and regression testing • Alpha, beta and acceptance testing • Software testing and quality assurance tools. Open source testing using NUnit and JUnit. • Software testing and QA project • Planning, organizing, and budgeting test procedures • Cost management of test projects: cost estimating, cost budgeting, project financing, cash flow management • Time management of test projects: schedule development, resource planning and allocation, network diagrams • Project quality management: standards, documentation, objectives, planning and control • Human resource management: QA project HR planning, staff acquisition and deployment, team building and training, team management • Managing outsourced QA projects: virtual team management, offshore team management





Video

https://youtu.be/xCwkjZcEK6w

Important

Testing of SW

- **QA –quality assurance or SW testing**: in narrow definition means ensuring that program performs according to some specs (this does NOT mean the specs are correct nor that the SW is usable) ➔ **focus of this class**
  - **Performing to specs**: the program does what is specified AND it has no bugs. Note: it can have no bugs but still miss on specs

Usability testing is <u>different</u> from QA, it is testing to ensure that the SW is usable for intended users, not to find bugs

- **Performance testing**: program is tested for speed, load etc.

Difference between usability testing and QA

Important

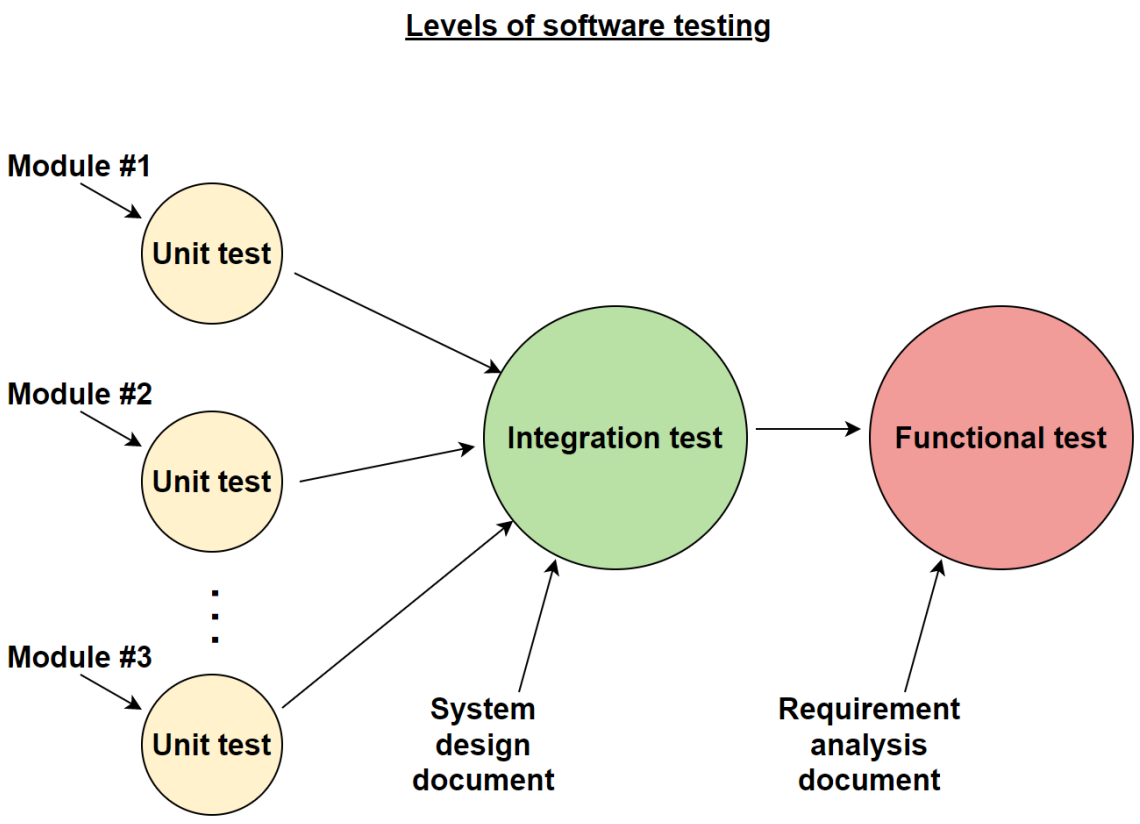
- **Usability testing**: check if the SW is easy to use
  1. Give user a task to do describing only **WHAT** to do not HOW to do it ("find an emergency situation in one of the counties in California")
  2. Collect input on how user feels after doing the task ("finding an extreme weather condition was easy" – strongly agree | x | x | x | strongly disagree)
- **QA testing**: check if SW performs to specs
  1. Give user very specific tasks on WHAT to do ("enter/select 'Health' in search field and verify that you get COVID or health infections result items located in specific counties in California"
  2. Tester reports PASS or FAIL

Important

More on definitions

- **SW testing** (often called QA) – ensuring SW performs to specs. Narrower than SQA
  - http://en.wikipedia.org/wiki/Software_testing
- **SW Quality Assurance (SQA)** is broader that SW testing and refers to the whole SE process
  - http://en.wikipedia.org/wiki/Software_Quality_Assurance
- **SW Quality Control (SQC)** involves checking all the SE documentation (requirements, specs etc.) – refers to the product while SQA refers to the process

Levels of Software Testing



Levels of software testing

Important

Most important types of testing

- Functionality testing – make sure functions do what they are supposed to do
  - Unit testing – done by individual programmers before checking in
  - Smoke testing – quick automated test (used as part of daily build and test)
  - Comprehensive or regression testing – longer, automated, does complete test
- Cross platform testing – browsers, OSes, libraries
- Usability testing – a) General usability; b) UI functionality
- Load/performance testing – how does the system perform under load (applications, users, data size etc.)
- Security testing – can system withstand malicious attacks and code
- Certification/compliance testing –in certain mission critical areas

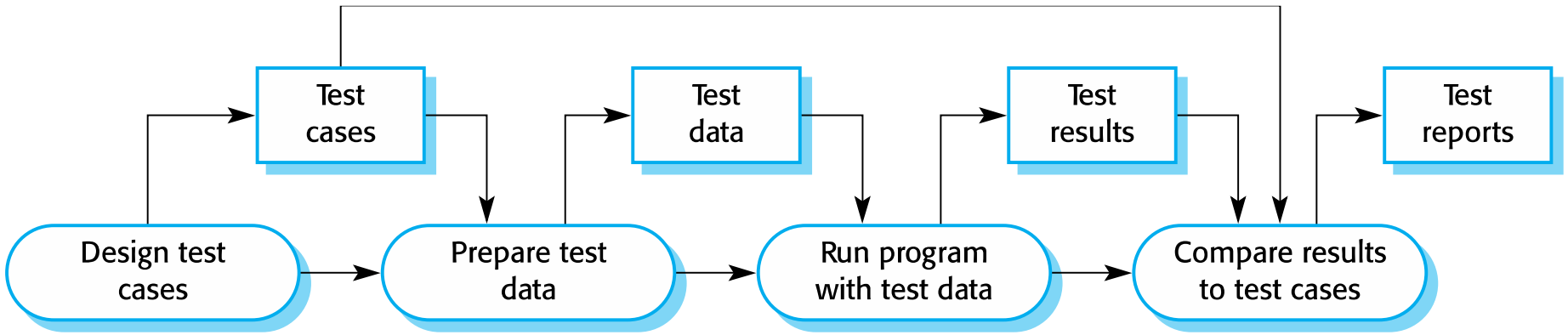
Important

Most important types of testing

- Unit testing: tests individual developer performs on the "unit" he she developed before submission (can be automated too)
- Smoke testing: quick test (often done daily as part of automated build and test) to quickly check if the system is in reasonably good state (compiles, builds, runs, performs basic tasks)
- Comprehensive test: tests all functions, may take long time
- Regression testing: used repetitively to test the whole system after some changes were made
- Integration testing: used when code is developed by many teams, must budget time to test integrated code after it has been tested as a unit. Important for Open Source and in global SE

A model of the software testing process

Important



Kinds of bugs

Important

- Bugs happen repetitively, upon same conditions – ***repetitive bugs*** (easy to diagnose…)
- Bugs happen from time to time – conditions not clear – *intermittent bugs* (hard to diagnose)
- System does not conform to some standard – *certification testing*
- System is too slow – *performance testing*
- System gives different results in different OSes – *cross platform compatibility testing*
- UI behaves differently for different browsers - *cross browser compatibility testing*
- Security bugs/exposures – enable malicious attacks and code to cause damage and security breaches – *security testing*

Important

Types of bugs - behavior

- **Repeatable**: always happen when certain specs are followed (easier to diagnose)
- **Intermittent**: one can not find consistent ways to generate the bug (hard to diagnose)
- **Performance**: way out of specified performance
- **Racing/timing**: system works with slow input, but not with fast input or speed

Important

Bug management

- Bugs need to be categorized, prioritized and tracked (besides being squashed)
- Agree and communicate bug categories to all (QA and Development)
  - Bug <u>severity</u> categories – external view of bug's overall *impact*
    - Critical, serious, minor
  - Bug <u>priority</u> – *when* they need to be dealt with
    - Test blocker, show stopper, beta stopper, release stopper, regular, deferred
- Have an <u>easy to use and consistent</u> system accessible to Development and QA

Classifying Bugs – Severity - Impact on system performance

Important

- CRITICAL
  - crash, halt, hang
  - loss of data
  - major functionality missing / in error
  - security violations
- SERIOUS
  - performance issues
  - functionality
  - many migration issues
  - ease of use
- MINOR
  - spelling errors
  - minor functionality issues

Classifying Bugs – Priority – *when* the bug needs to be fixed

Important

- *Test Blocker* – tests can not be done until fixed
- *Show Stopper* – the whole product in jeopardy
- *Beta Stopper* – must fix before beta is launched
- *Release Stopper* - must fix before release
- *Post-release* – OK to fix after release
- *Deferred* – do it later
- *Document* – (and don't fix)

Important

Testing strategies

- *Black box*
  - No knowledge of internals.
  - Test plans derived from specs and requirements
  - Testing done by running the executable, no view of the source code
- *White box*
  - Access to source code is available
  - Test derived from specs and also from inspecting the code (e.g. problem code is tested)
  - Test done by running the code and inspecting the source code

Testing strategies

- Developers do *unit testing* of their module locally, on their system (necessary but not sufficient)
- *Code reviewed* before checking in
  - Developers optionally can add test suggestions for QA
- Code integrated, QA tests the code again as well as the whole system - *integration testing*
- Integrated system rebuilt and smoke tested often – *build and smoke test*

Important

Testing strategies

- How to maximize coverage with minimal number of test sequences
  - Partition inputs into equivalence sets or classes – each class member executes single path, so only a few elements of each class need to be used for testing
  - Exhaustive testing: test all combinations of inputs and conditions (very expensive, possible only in some cases)
  - Path testing: test all paths
  - Code coverage: cover all code or all key code
  - Integration testing: Test modules independently, then test full system by incrementally adding modules

Important

Test Plan – outline

1. Test objectives: what is tested and what is not tested
2. Setup of HW
3. Setup of SW including at what SW state the testing beings
4. Test environment, human resources needs, expected time to complete
5. Risks, contingencies
6. End report format requirements
7. Actual test sequence, test input AND test expected results
8. Final report

Important

Examples of WRONG test plan

- **Input**: enter “red”
- **Output** : Check that results are correct

- **Input**: enter some input
- **Output** : Check that results are correct

- **Input**: enter “red”
- **Output** : Check that ALL results contain “red”

Important

Examples of GOOD test plan

- **Input**: enter “red”
- **Output** : Check that you get 5 results, all containing string “red” in name field

- **Input**: enter “red”
- **Output** : Check that results consist of exactly these
  - Result 1
  - Result 2
  - -…
  - Result 5

Important

Report of tests – make them easy to read (there are a lot of test cases)

- Tabular format

Make this **specific**

Number	Description	Test input	Expected output	PASS/FAIL
1	Test % like In search for name field	"red"	Get 5 results, all have "red" in name field	

Important

How to present QA status and results

- Graphs for QA progress reports
  - Number of open bugs in time
  - Number of bugs in time grouped by area
  - Number of bugs in time grouped by severity/priority
- Critical in making product release decisions
- Has to be easy to understand to non-CS management

Important

Usual testing sequence

Daily or very often

1. Developers do unit testing
2. Submit code for review
3. Reviewed code OKed (or fixed) and submitted to source control
4. Daily “smoke test” or full test done
5. a) move on to next submissions or b) lock submissions and fix problems from last working build

QA department performs scheduled full tests

New tools like Jenkins, Travis can integrate these steps

Unit testing tools

- One example: junit
  - http://en.wikipedia.org/wiki/JUnit
  - http://junit.org/
- Can write your own
- Relates to “test driven development” – write test plan before coding, then test it

Important

Smoke test

- Efficient test that can be applied quickly, in an automated way, to do some basic test (system compiles, files can be stored and retrieved etc.)
- Intended to discover obvious bugs and problems quickly
- Used as part of “build and test” process to be applied often (even daily)
- By having this, one limits number of possible bugs to a small number (with high probability only since the last some and test)
- Make it as comprehensive as efficiently allows, automate and integrate with build system with automated report generation and possibly e-mail alerts
- Critical for effective SW development

Important

How to test UI subsystems in QA context (adhering to specs, NOT for usability)

- To test UI one has to try all the sequence of user actions to cover all the functions and “buttons”
- Very time consuming ➔ automation tools are developed
- UI test automation tools:
  - QA set up the system
  - QA trains the system manually by providing user input (e.g. clicks on buttons) and expected results
  - QA test tool records the training data above and repeats during the test
  - Test results are recorded and summarized

M4-5

- **Milestone 4.** Beta Launch, QA and Usability Testing and Final Commitment for Product Features (P1 list). November 12th 2024 (W12)

- **Milestone 5.** Final Demo and Documents for Grading. December 10th 2024 (W15)

Final Project Main Functions

- <u>Web App: Voting Guide with AI+</u>

1. **Search & Filter** by state, political party and issues/policies.
2. **Enter** official information (political parties, voter registration, polling location, voting process, etc.)
3. Site **Registration** (College Students, General public, Officials, Admin, etc.).
4. **Rate** candidates.
5. Search results may be displayed **using maps**
6. **AI capabilities** is a plus.

Two entrepreneurs, SFSU alumni, decided to create a start-up comprising of a team of about 6+ students to develop a web-based service to help USA college students and general public to identify *county, state, and federal*/national **candidates** running for public offices and their respective *policies*, and *background*. The web application will allow officials from the different **political parties** to enter the corresponding official information, and the **Elections office** to enter information about voter registration, polling locations, and voting process. College students and general public should be able to search by state, political party and issues/policies so results can list the corresponding data, and at the same time; user (student and general public) can enter their issues of interest and the AI chatbot can assist them with mapping what candidates are aligned with the user interest. This application is targeting three primary audiences: (1) College Students, (2) General Public, and (3) Officials from the political parties and election office. Students and general public users should be able to rate candidates. Officials from political parties and election office may need to be registered to use all the functionality of the application. General users may access to limited search capabilities and application functionality. Search results may display maps.

Study Materials

Acknowledge

- Content of these slides are based on course materials created by **Dr Dragutin Petkovic**

Some major problems happened due to poor SW QA

- http://www.softwareqatest.com/qatfaq1.html#FAQ1_3

QA principles

- Same principles and high level process apply to all platforms from servers to PCs to mobile
- Differences are in specific implementaion, tools etc.

Bugs/errors examples

- Specs not followed (function missing or function behaves not as to specs, otherwise application looks OK, no bugs)
- System crashes
- System hangs
- Computation done wrong
- System inconsistent – upon repeating the same steps one gets different results or behavior
- System works initially but after a while shows problems
- System works with manual input but not with programmatic (fast) input
- System course problems with “corner cases”
- System does not read/store required file formats
- Loss of data

Important

Bug management (2)

- Use bug tracking systems to record/manage bugs:
  - Bug Severity
  - Bug Priority
  - Description
  - Assignment (who is to look into ti - determine from list of assigned developers)
  - Status (open, in progress, closed, deferred)
- Have teams assigned to categorize and analyze bugs. Often, many bugs come from one single defect. Priorities and categories sometimes need to be addressed (and changed) by team of developers and QA
- Bugs can be opened by anybody, closed <u>only</u> by QA
- Late in product cycle, frequent (daily) meetings of bug teams (QA, development)

Important

Bug management (3)

- Bug reports:
  - Total number of bugs in time (use charts to present)
  - Number of bugs in each category, priority, subsystem, developer (optional)
  - Analysis of causes, trends, critical issues (note: bugs can be correlated – some bug can cause many others…)

- Final Go/NO GO on product release usually gated by QA report summarizing above statistics

Test strategies for SW

- How to define and discover bugs

Recall: QA job is to stress test the system and diagnose as many problems as possible with minimal resource/time

The earlier problems are discovered, the cheaper it is

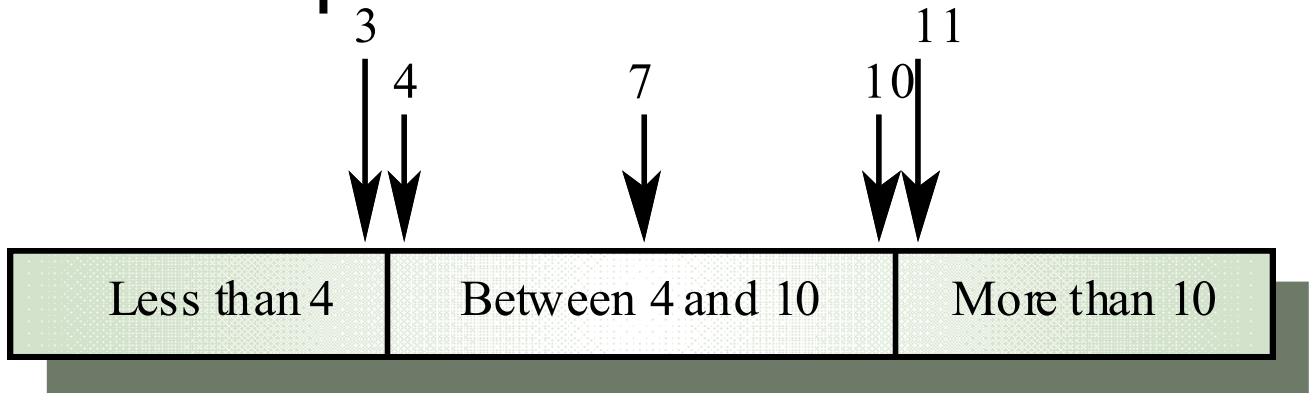
Important

Test strategy 1: Equivalence partitioning

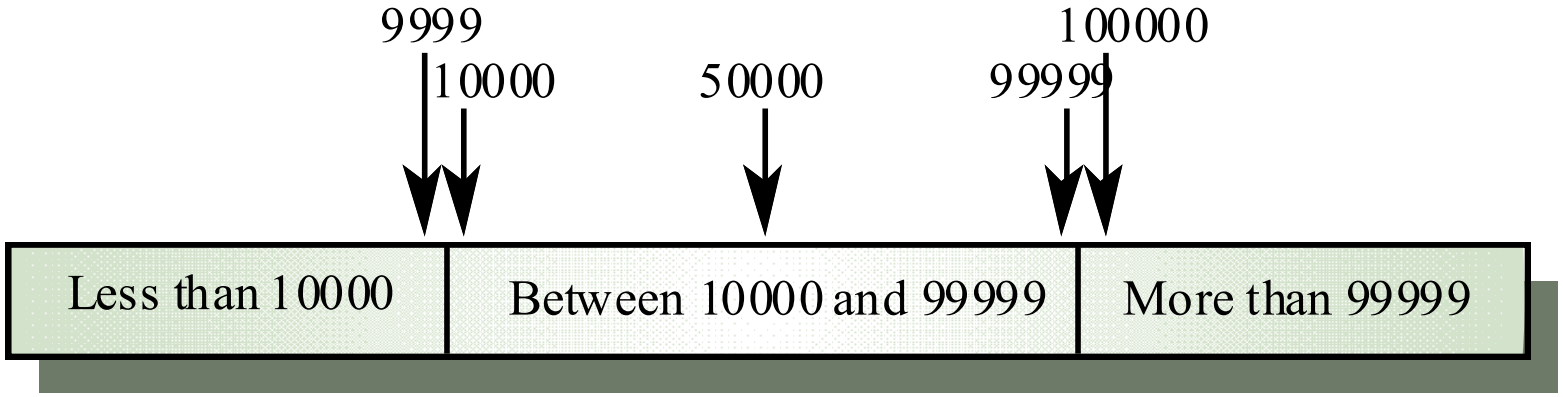
- Partition system inputs and outputs into 'equivalence sets'
  - If input is a 5-digit integer between 10,000 and 99,999, equivalence partitions are <10,000, 10,000-99, 999 and > 100, 000
- Choose test cases at the boundary of these sets
  - 00000, 09999, 10000, 99999, 100000, 100001

From: Ian Sommerville

Equivalence partitions



Number of input values



Input values

From: Ian Sommerville

Examples from WWW forms

- Specs:
  - User name – text entry. Validate that entered string has more than 1 characters
  - User phone – text entry. Validate that the string has at least 7 digits
  - User age – text entry. Validate that the value is greater or equal 10 and less than 100
- What are equivalence sets here?

Binary search - equiv. partitions

- Key element in array
- Key element not in array
- Input array has a single value
- Input array has an even number of values
- Input array has an odd number of values

```
class BinSearch {

// This is an encapsulation of a binary search function that takes an array of
// ordered objects and a key and returns an object with 2 attributes namely
// index - the value of the array index
// found - a boolean indicating whether or not the key is in the array
// An object is returned because it is not possible in Java to pass basic types by
// reference to a function and so return two values
// the key is -1 if the element is not found

    public static void search ( int key, int [] elemArray, Result r )
    {
        int bottom = 0 ;
        int top = elemArray.length - 1 ;
        int mid ;
        r.found = false ; r.index = -1 ;
        while ( bottom <= top )
        {
            mid = (top + bottom) / 2 ;
            if (elemArray [mid] == key)
            {
                r.index = mid ;
                r.found = true ;
                return ;
            } // if part
            else
            {
                if (elemArray [mid] < key)
                    bottom = mid + 1 ;
                else
                    top = mid - 1 ;
            }
        } //while loop
    } // search
} //BinSearch
```

Test strategy 2:
White-box testing:
Test cases derived
Upon code inspection

Binary search (Java)

From: Ian Sommerville

Important

Test strategy 3: Path testing

- The objective of path testing is to ensure that the set of test cases is such that <u>each path</u> through the program is executed at least once
- The starting point for path testing is a program flow graph that shows nodes representing program decisions and arcs representing the flow of control
- Statements with conditions are therefore nodes in the flow graph

From: Ian Sommerville

Important

Test strategy 4: Integration testing

- Goal: Tests complete systems or subsystems composed of integrated components
- Critical when systems are build from components, by different teams (most often the case)
- Each subsystem is first unit tested
- Subsystems are then incrementally added to the system and tests are performed at each step

Any considerable system should have well identified modules or subsystems ➔ it allows "divide and conquer" strategy in testing (and development)

Test Plans

- QA generates test plans
- Purpose:
  - Instructions for testers
  - Planning, resource allocation

Test sequence

- Intent is to be clear and complete, and require minimal user experience (deferent from usability test plans)
- Create test plan/sequence (e.g. instructions) with <u>specific</u> input and <u>specific</u> output so PASS/FAIL can be easily (and automatically) determined
- Cover + and – cases and corner cases

Examples of test sequences - search

- Test sequence: “Input “red” and check output to be correct”– WRONG sequence/plan – why?
- Correct approach
  - Establish “ground truth” e.g. test data ➔what is the <u>specific correct output for certain specific input</u> (also helps in Test Driven Development)
  - Use specific <u>concrete</u> cases for test plan/sequence

Examples of test sequences – search (2)

- Assume your search is by CATEGORY (via pull down) and text entry (using %LIKE)
- Create your own test DB
  - Item 1: category = FURNITURE"; description = "new red sofa"
  - Item 2: category = FURNITURE"; description = "new blue sofa"
  - Item 1: category = FURNACES"; description = "new red"
  - Item 1: category = FURNACES"; description = "goes with red furniture"
- Develop test cases to test + and – and corner cases. Try to "break" the system by choosing corner cases that "stress the system"
- Determine "by hand" correct answers
- Use the above to determine test plan/sequence and expected correct output

Examples of test sequences – search (3)

- Some examples of corner cases for the test DB
  - Category = CARS
  - Category = FURNITURE text = “red”
  - Category = FURNITURE text = “fur”
  - Category = FURNACES text = “red”
  - Category = ALL text = “red”
  - Category = ALL text “”
  - .....

Resources

- QA Resources (definitions, tools., examples, papers):
  - http://www.aptest.com/resources.html
  - http://www.softwareqatest.com/

  - One example of the tool to track and manage QA: Bugzilla http://www.bugzilla.org/

Testing of UI systems

Popular tools for testing PC UI subsystems (no endorsements, just some examples)

- Selenium http://seleniumhq.org/
- Qt Group https://www.qt.io/product/quality-assurance?gclid=CMGzuYWRpawCFQUaQgodTG1DDA
- Try it! Use it!
- Good for job search – many companies use it and require it..

Resources: Other relevant topics

- Continuous integration and tools to autmate it – Jenkins, Travis
- WWW testing
- Resource leaks testing (e.gt. Memory leaks)
- Mobile testing
- Cross browser testing
- Performance testing

Continuous Integration (CI)

Jenkins

- Provides the link between committing and deploying.
- Allows monitoring by management and tech leads
- After commit to repo, Jenkins *automatically* runs tests:
  - If FAIL: Jenkins generates reports, rolls back codebase to last working version
  - If SUCCESS: Jenkins deploys to testing server
  - Status reported and logged
- https://jenkins.io/

Continuous Integration (CI)

Jenkins

- Example PHP tests that Jenkins would run:
  - PHP Unit - Unit tests for PHP code
  - PHP LOC - Measures lines of code, comments, lines in classes and methods
  - PHP Depend - Static code analysis on a given source base.
  - PHP Syntax checker
- Can be configured & extended for other tests & behaviors

Travis tool for continuous integration

- https://en.wikipedia.org/wiki/Travis_CI

"When Travis CI has been activated for a given repository, GitHub will notify it whenever new commits are pushed to that repository or a pull request is submitted. It can also be configured to only run for specific branches, or branches whose names match a specific pattern. Travis CI will then check out the relevant branch and run the commands specified in *.travis.yml*, which usually build the software and run any automated tests. When that process has completed, Travis notifies the developer(s) in the way it has been configured to do so[5]—for example, by sending an email containing the test results (showing success or failure), or by posting a message on an IRC channel. In the case of pull requests, the pull request will be annotated with the outcome and a link to the build log, using a GitHub integration."

Important

QA for Mobile platforms

- Similar principles and polices, different tools
  - http://en.wikipedia.org/wiki/Mobile-device_testing
  - http://developer.android.com/tools/help/monkey.html
  - http://www.vogella.com/tutorials/AndroidTesting/article.html
  - http://www.softwaretestinghelp.com/robotium-tutorial-android-application-ui-testing-tool/

Important

Web Testing

- Testing of Web applications
- Wikipedia http://en.wikipedia.org/wiki/Web_testing
  - Functionality testing – make sure functions do what they are supposed to do
  - UI testing – a) usability; b) UI functionality
  - Load/performance testing – how does the system perform under load (applications, users, data size etc.)
  - Security testing – can system withstand malicious attacks and code
  - Compatibility testing (cross browser support)

Important

Cross platform testing and issues - PCs

- Systems can behave differently and produce different results with different Oses (but produce no bugs)
- Often, issues exist with floating point library, round of errors etc. – done differently in deferent OSes
- Critical to test for numerical and scientific applications
- Programming strategy and arch. have to enable *portability*
- Need to test for this – do not assume it
- Test matrix grows fast: # of OS X # of browsers
- Use VMs or Cloud computing to efficiently cover combinations OS/browsers (more on cloud later)

Important

Cross browser testing and issues

- Remains a serous issue despite promises to the contrary ("program once, run on all browsers")
- 3-4 major variants
- Need to support at least 2 versions of each
- iPAD, mobile platforms also need to be addressed

- Cross browser compatibility:
  - Web application has to <u>work</u> on all browsers (at least 3-4 major variants) and increasingly on mobile and iPADs
  - Web application has to create the same <u>UI experience</u> for different browsers

Important

Cross browser testing and issues (2)

- Wikipedia
  - http://en.wikipedia.org/wiki/Cross-browser
- Exponentially adds to number of test cases
  - Use Web UI test automation
    - http://seleniumhq.org/
    - http://www.froglogic.com/?gclid=CMGzuYWRpawCFQUaQgodTG1DDA
- Strategies:
  - Adhere to cross browser compatibility in coding and design
  - Use simple functions and approaches in coding, they usually are easier to work on all platforms
  - Use latest tools to test your browsers (reviews are here
    - http://www.smashingmagazine.com/2011/08/07/a-dozen-cross-browser-testing-tools/
    - http://slodive.com/web-development/cross-browser-testing/)
- Mobile access – often needs different UI design and coding

Testing in Open Source world

- Who wins?
  - Large companies with well established QA departments or
  - Open Source community which continually tests the code?
- Open source wins in general
- Since you will use many components from Open Source integration testing is the key plus white box testing – check the code too

Other types of testing

Important

Testing for memory leaks, handle leaks, racing conditions (main cause of *intermittent* bugs)

- Programmers often make mistakes in not releasing resources (memory, handles)
- Systems can work when tested briefly, then slow down or hang up after long operation. Problems are hard to trace without tools – usually there are no obvious classic bugs.
- Tools help point to problems
  - Valgrind http://valgrind.org/
  - SIMPLE: Use Windows task manager or similar – run the program in the loop and observe memory usage
- Use stress testing, preferably automated

Important

Performance testing and tuning

- Various tools (I.e. profilers) can help give an idea where the time is spent in the SW
- Often, most time is spent in very few modules. One more reason to have the code well modularized ➔ you can easily focus your testing on key "suspects"
- Extremely important in SW design and development: use on prototypes and final systems
  - Make sure the benchmark itself does not interfere with the application
  - Make sure system conditions and setups are realistic (memory, other processes running, network, caching, load time…)
  - Repeat measurements many times and average results
- Develop and test first for correctness, then speed

Performance measurements

- Measure efficiency with given load (e.g. other applications, numbers of users) or system constraints (e.g. memory, network)
- Refers to I/O, CPU, graphics, DB, throughput, number of users
- Standard benchmarks are used in order to measure efficiency – they simulate realistic loads
- Good benchmarks are run by independent companies, in carefully controlled and published settings
- Benchmarks help compare apples with apples
- Make sure to also test the actual application you have to provide to the user

Benchmarks – good resources and organizations

- Valgrind http://valgrind.org/
- Standard Performance Evaluation Corp (SPEC)
  - http://www.specbench.org/
- TPC – Transaction performance Council (for databases)
  - http://www.tpc.org/
- Web benchmarking (user load, database connections etc…)
  - Jmeter http://en.wikipedia.org/wiki/JMeter
  - LoadRunner – test for loads http://en.wikipedia.org/wiki/LoadRunner

Important

Security QA

- Usually consists of:
  - White box testing of code (must see code, example: array bounds checking) (Note: good to follow "safe coding style" to prevent security problems…)
  - Audit of architecture:
  - Black box testing: trying to break it;
  - Fault injections
  - Audit of policies and procedures (e.g. password administration, personnel security checks, training, execution of polices etc.)
- (Check slides on SE for Security)

Advanced topics – QA using Virtual machines

Important

- Testing using Virtual machines (I.e. VMware) solves several key problems:
  - Offers all HW/SW/OS/Browser combinations via SW provisioning
  - Easy to revert to initial state
- Can run multiple virtual machines on one PC – speeds up QA tremendously
- Can automate QA by 1) launching Virtual Machines (VM) with reconfigured test programs, 2) perform the tests in each VM, 3) collect the results
- VMs from Open Source offerings, also www.vmware.com
- Resources on use of VMs in QA:
  - https://www.howtogeek.com/206286/stop-testing-software-on-your-pc-use-virtual-machine-snapshots-instead/
  - https://www.apriorit.com/qa-blog/223-virtualization-in-testing

Important

Using Cloud computing for QA

- Similar to using VMs, but in this case you “rent” and “provision” all VMs via cloud. Cloud is provided and maintained by others and provides all combinations of virtual HW/SW/OS you need
- Strongly recommended – much cheaper and faster compared to having the right VMs and HW
- Resources:
  - http://www.softwaretestinghelp.com/getting-started-with-cloud-testing/

Important

QA and outsourcing

- Note successful outsourcing of some QA – allows for non stop development – E.G. build, compile in USA, send for test to India; next day check test results and continue
- What QA is easy to outsource: components, subsystems etc.
- Integration testing – usually done at the central/owner site
- Usability testing – best done close to the customer base (but with Internet this loses its “location” sensitivity)

QA management strategies

Important

- Develop test plans together with product team
- Budget enough time for QA
- Insist on proper code review and unit testing before integrating into full system
- Establish use of some modern tool for bug management (e.g. Bugzilla) and agree on policies, bug categories and assignments. Enforce it.
- Use source code management to number releases – helps in regression testing
- Automate build and smoke test system into one, do it often, daily at the final stages of product cycle
- Do full and regression testing often
- Develop easy to read and comprehend test reports ➔ critical for making a decision on product launch

Future: QA in the era of on-line services and big data

Important

- Products not released is well orchestrated downloads but continuously available and modified (e.g. on-line services, on-line search)
- QA is augmented by “test-in-production” where QA is being continuously done
- Data scientists and ML experts analyze and discover/predict bugs

Great paper to read and discuss - M. Kim et all: Role of data scientist in SE teams", ICSE 2016

- Paper
  - http://homepages.dcc.ufmg.br/~figueiredo/disciplinas/2016b/sqm/paper10laerte.pdf
- Problems data scientists in SE work on:
  - Performance Regression
  - Requirements Identification
  - Fault Localization and Root Causes Analysis
  - Bug Prioritization
  - Server Anomaly Detection
  - Failure Rate Estimation
  - Customer Understanding
  - Cost Benefit Analysis

QA and Software Engineering, Teamwork and Process issues

- Involve QA early, as soon as requirements and specs are done
  - QA gets trained and prepares resources
  - Establishes teamwork
- Keep working closely with QA especially at the end of the cycle
- Let developers suggest test plans, but let QA do and own the final ones

Software Engineering, Teamwork and Process issues (2)

- While submitting code for review, outline test cases in separate section. This section should be sent to QA
- Cover test cases you have done and test cases that need to be done by QA
- Be sure to point out to possible weaknesses or issues or suspected problems in your code ➔ be open about it!

Software Engineering, Teamwork and Process issues (3)

- Build and smoke test often, daily at the end of the cycle
- Invest in automated test development (homegrown or commercial)
- Number releases and keep them, so you can go back in case some bugs appear
- Meet often with QA, daily at the end of the cycle

Remember, QA has usually the last word on whether the product is launched

QA as a career

- Excellent entry jobs in QA
- Helps you learn about products, later can go to support or development
- Involves good planning, scripting, QA automation, organization and management
- Often the key QA is kept in USA (final QA, integration QA etc.)

Review Questions

- Define SW quality as you see it
- Know the difference between SW testing (QA), SQA, SQC
- Different types of testing: know to define them and when and how to use them:
  - white box, black box
  - unit testing,
  - UI testing,
  - usability testing,
  - UI based testing
  - API testing
  - system testing
  - Scenario/usage testing,
  - regression testing,
  - integration testing,
  - customer application testing,
  - inter-operability testing,
  - benchmarking,
  - Certification
  - L10N/I18N testing
  - Beta testing

Review Questions

- Explain the meaning of the following bug classification of severity: critical, serious, minor
- Explain meaning of the following bug priority classification: test blocker, show stopper, beta stopper, release stopper, differed
- What are the sections in a typical test plan
- What is defect tracking and how the defects look at the beginning vs. at the time product is getting ready

Review Questions

- What is equivalence set partitioning and how is it used in testing
- What would be equivalence set partitions for testing the text input on the following:
  - Age is a number greater than 15 and less than 100.
  - What other inputs would you use to stress test the system
- WWW product has to be tested for compatibility with Wun2K (SP1, SP2, SP6), XP (SP1, SP2) and with IE V5, 6, and Mozila. How many test combinations QA has to do here.
- How is integration testing done?

Review Questions

- What is cyclomatic complexity on a program flow graph. Be able to compute it from the given graph. What is its significance in developing test cases
- Is it recommended to first focus on speed or correctness in developing SW
- Why should you involve QA in your SW development? List some benefits and also comment on when would you start involving QA in your SW development process

Tricky bug examples

1. Tried testing, it was OK. After running for long time, the system hangs up. Upon reload, it runs OK again, then hangs up
2. Intermittent errors, random results on output in case of error
3. Runs OK. Program does NOT use networking explicitly. When network is slow or down, it hangs up.
4. I tested it via UI, works OK. I tested it using test program and APIs, and it does not work reliably.

Tricky bug examples (answers on possible cause)

1. Resource leaks? (memory, handles). Use Purify from IBM/Rational. Check task manager for memory usage
2. Is everything initialized? Inspect the code. Arrays are not necessarily set to 0 by the compiler.
3. What is OS doing? Microsoft roaming feature? Is OS doing some of its own “hidden” work via networking? (You would be surprised…)
4. Possible race conditions. Inspect the code for problem spots. Linearize the code, put semaphores and barriers and rerun. Use stress testing.